

Project Report

CSP 1602: Software Engineering Lab

The MoodShelf

Intelligent Mood-Based Book Recommendation System

Submitted by

Richa Ray

Roll No.: BT23CS013

Under the guidance of

Dr. C Lalengmawia

Assistant Professor, Department of CSE



National Institute of Technology Mizoram

Department of Computer Science & Engineering

Aizawl – 796012, Mizoram, India

April 2026

ABSTRACT

TheMoodShelf is an intelligent, mood-aware book recommendation web application that leverages natural language processing and semantic similarity to connect readers with literature that resonates with their current emotional state. Modern readers often struggle to find books that match not just their genre preferences but their moment-to-moment feelings — a gap that existing recommendation platforms, which rely primarily on ratings and browsing history, fail to address.

This project addresses that gap by accepting free-text mood input from the user, classifying the underlying emotion using the DistilRoBERTa transformer model fine-tuned on seven emotion categories, and mapping the detected mood to a curated set of genre affinities. Candidate books are then ranked by cosine similarity between the user’s input and pre-computed sentence embeddings (all-MiniLM-L6-v2) of book descriptions, ensuring results are semantically relevant rather than keyword-matched. The system additionally supports mood journaling, a personal saved-books library, an analytics dashboard with mood distribution charts and activity trends, and a personalisation layer that boosts genres historically preferred by the user for a given mood.

The application is built on a Python/Flask backend with a MySQL database, and is deployed publicly on Hugging Face Spaces using a Docker-based pipeline, accessible at <https://resincinnamon-themoodshelf.hf.space>. Evaluation through manual testing and live usage demonstrates accurate mood classification across a wide range of emotional phrasings, including negated expressions and low-confidence edge cases handled by keyword overrides. TheMoodShelf offers a novel, human-centred reading experience that bridges affective computing and literary discovery.

Keywords: *mood detection, book recommendation, DistilRoBERTa, sentence transformers, affective computing, Flask, Hugging Face Spaces*

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. C Lalengmawia, Assistant Professor, Department of Computer Science and Engineering, NIT Mizoram, for his invaluable guidance, encouragement, and constructive feedback throughout the development of this project. His insights on software engineering principles and system design greatly shaped the architecture and quality of TheMoodShelf.

I am also thankful to the Department of Computer Science and Engineering, NIT Mizoram, for providing the computational resources and academic environment that made this project possible.

Special thanks to the open-source community — particularly the Hugging Face team for the Transformers library, the sentence-transformers project, and the Flask community — whose tools formed the backbone of this system.

Richa Ray

Roll No.: BT23CS013

April 2026

TABLE OF CONTENTS

Chapter 1: Introduction

- 1.1 Background
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope and Limitations
- 1.5 Organisation of the Report

Chapter 2: Related Work

- 2.1 Existing Systems and Approaches
- 2.2 Tools and Technologies Reviewed
- 2.3 Gap Identified

Chapter 3: Methodology

- 3.1 System Architecture
- 3.2 Requirements Analysis
- 3.3 Design
- 3.4 Implementation
- 3.5 Testing Strategy

Chapter 4: Deployment

- 4.1 Hosting Platform
- 4.2 Database Hosting
- 4.3 Deployment Pipeline
- 4.4 Environment Configuration
- 4.5 Accessing the Application

Chapter 5: Results and Discussion

- 5.1 Environment and Setup
- 5.2 System Screenshots and Outputs
- 5.3 Discussion

Chapter 6: Conclusion

- 6.1 Summary
- 6.2 Future Work

References

Chapter 1: Introduction

Reading has always been a deeply personal activity, shaped not just by literary taste but by the emotional landscape of the reader at any given moment. Yet most digital book recommendation systems treat emotion as an afterthought — they suggest books based on past ratings, bestseller lists, or collaborative filtering from other users, without ever asking how the reader feels right now. TheMoodShelf is built on a different premise: that the best book for a person today is the one that meets them where they are emotionally.

1.1 Background

Affective computing — the field concerned with systems that recognise, interpret, and simulate human emotions — has matured rapidly with the advent of large pre-trained language models. Transformer-based architectures such as BERT, RoBERTa, and their distilled variants can classify emotion from short text inputs with high accuracy, opening a practical pathway to emotion-aware applications in consumer software.

Simultaneously, semantic search has replaced keyword matching as the dominant paradigm for information retrieval. Sentence Transformers produce dense vector representations of text that capture meaning rather than surface tokens, enabling similarity comparisons between heterogeneous pieces of text — such as a user’s mood description and a book’s summary. TheMoodShelf combines both advances: transformer-based emotion classification to understand the user’s state, and semantic embedding similarity to find books whose content and tone resonate with that state.

1.2 Problem Statement

Existing book discovery platforms such as Goodreads, Amazon, and Open Library rely on genre tags, star ratings, or collaborative filtering. None of these mechanisms account for a reader’s current emotional state, which research consistently identifies as a primary driver of reading enjoyment and retention. A reader who is anxious before an exam needs different material than the same reader in a calm, exploratory weekend mood — yet today’s systems do not distinguish these contexts.

Furthermore, no lightweight, open-source system currently integrates mood journaling, long-term mood analytics, and personalised book recommendations in a single web application accessible without specialised hardware or accounts on third-party platforms.

1.3 Objectives

- To develop a web application that accepts free-text emotional input from users and classifies the underlying mood using a transformer-based NLP model.
- To implement a semantic book matching engine that ranks books by their contextual relevance to the user’s input using sentence embeddings.
- To build a personalised recommendation layer that learns user genre preferences over time and adjusts rankings accordingly.
- To provide a mood journaling feature that allows users to log and revisit their emotional states alongside reading history.
- To create an analytics dashboard that visualises mood trends, emotion distributions, and reading activity over time.
- To deploy the complete system publicly on Hugging Face Spaces using a Docker-based pipeline.

1.4 Scope and Limitations

TheMoodShelf covers mood classification across eight mood categories (happy, sad, stressed, calm, anxious, lonely, tired, motivated), book recommendations from a static dataset of 998 titles, user authentication, session-based state management, and a MySQL-backed persistent store. The application is designed for desktop browsers and is deployed on Hugging Face Spaces.

Known limitations include: the book dataset is static and does not update in real time; the dataset contains no author information; language support is limited to English; and the emotion model may misclassify highly ambiguous or ironic text. The free hosting tier on Hugging Face Spaces imposes cold start delays of approximately 60–90 seconds after periods of inactivity, and the free MySQL database (FreeSQLDatabase.com) has a 5MB storage ceiling. The personalisation layer requires sufficient user history to produce meaningful boosts.

1.5 Organisation of the Report

Chapter 2 reviews existing mood-based recommendation systems and the NLP tools they employ. Chapter 3 describes the full methodology: system architecture, requirements analysis, design, implementation, and testing strategy. Chapter 4 documents the complete deployment process. Chapter 5 presents results through annotated screenshots and discusses the system’s performance. Chapter 6 concludes with a summary of contributions and directions for future work.

Chapter 2: Related Work

This chapter surveys existing work in emotion-aware recommendation systems, sentiment-based information retrieval, and the NLP models that underpin them.

2.1 Existing Systems and Approaches

Liu et al. (2018) proposed a sentiment-enhanced collaborative filtering system for movie recommendations. By embedding user reviews with a CNN-based sentiment analyser and incorporating the sentiment vector into the matrix factorisation objective, they achieved a measurable improvement in root mean squared error over baseline collaborative filtering. However, their approach still relies on historical ratings and cannot produce recommendations for new users or novel emotional states not previously encountered in the training corpus.

Nitu et al. (2021) explored emotion-based music recommendations using facial expression recognition to detect mood in real-time and map detected emotions to playlist attributes. While effective for music, the approach requires a camera and does not generalise to text-based mood input or book recommendations.

Hartmann et al. (2022) released the emotion-english-distilroberta-base model on Hugging Face, a DistilRoBERTa model fine-tuned on a large corpus of social media text annotated with six Ekman emotions plus neutral. The model is compact enough to run on CPU in a production web server, making it the practical choice for TheMoodShelf over larger alternatives.

Reimers and Gurevych (2019) introduced Sentence-BERT, demonstrating that BERT-derived sentence embeddings could be used for semantic textual similarity with orders-of-magnitude speedup compared to cross-encoder inference. The all-MiniLM-L6-v2 model produces 384-dimensional embeddings and achieves competitive benchmark scores while being fast enough for production use.

Moody (moodyreads.com) is a commercial product that allows users to select emotions from a preset grid and receive curated book lists. It does not accept free-text input and relies entirely on editorial curation rather than automated NLP, making it difficult to scale and unable to capture nuanced or mixed emotional states.

2.2 Tools and Technologies Reviewed

Several alternative architectures were considered during design. VADER and TextBlob, rule-based sentiment tools, were rejected because they only capture valence (positive/negative) and not the richer emotion taxonomy required. DistilBERT fine-tuned on GoEmotions (Demszky et al., 2020) was a strong candidate but produces 27 fine-grained emotion labels that complicate mapping to mood-genre pairs; DistilRoBERTa with seven labels was a better fit. For the embedding model, FAISS was considered for approximate nearest-neighbour search but was deemed unnecessary at the dataset's scale; NumPy cosine similarity suffices with negligible latency.

For deployment, Render and Railway were evaluated but ruled out due to insufficient free-tier RAM (512MB) for loading the ML models, which require approximately 2GB. Hugging Face Spaces with the Docker SDK was selected as it offers 16GB CPU RAM on the free tier and is purpose-built for ML model serving. FreeSQLDatabase.com was selected as the MySQL provider after Aiven's free MySQL tier was found to be unavailable due to high demand.

2.3 Gap Identified

The literature reveals that while emotion-aware systems exist in music and movie domains, a lightweight, open-source web application combining free-text mood input, transformer emotion classification, semantic book matching, mood journaling, and long-term analytics in a single integrated, publicly hosted product does not currently exist. TheMoodShelf fills this gap for the reading domain.

Chapter 3: Methodology

This chapter describes the end-to-end design and implementation of TheMoodShelf, from high-level architecture to individual algorithmic decisions.

3.1 System Architecture

TheMoodShelf follows a classic three-tier web architecture: a presentation tier (HTML/CSS/JavaScript Jinja2 templates), a logic tier (Flask application server with ML models), and a data tier (MySQL database). The application server loads two ML models at startup — the DistilRoBERTa emotion classifier and the all-MiniLM-L6-v2 sentence embedder — and pre-computes embeddings for all book descriptions in the dataset. This one-time cost at boot eliminates per-request embedding latency, reducing recommendation response time to under three seconds.

3.2 Requirements Analysis

Functional Requirements

ID	Requirement	Priority
F-001	User registration and login with securely hashed passwords (Werkzeug PBKDF2)	High
F-002	Free-text mood input with emotion classification across 7 emotion categories	High
F-003	Display top 5 book recommendations with similarity scores and metadata	High
F-004	Save books to a personal library with associated mood context	High
F-005	Mood journaling: write and view entries linked to mood sessions	Medium
F-006	Analytics dashboard: mood distribution, emotion bar chart, 30-day activity chart	Medium
F-007	Generate-again: regenerate recommendations avoiding previously shown books	Medium
F-008	Personalisation: boost genres historically preferred by user for current mood	Medium

Table 3.1 — Functional Requirements

Non-Functional Requirements

Category	Requirement	Target
Performance	Mood analysis response time	< 3 seconds
Performance	Page load time	< 2 seconds
Performance	Concurrent users supported	50+
Security	Password storage	Werkzeug PBKDF2 hashing
Security	Database query safety	Parameterised SQL
Security	Production transport	HTTPS (Hugging Face)

Reliability	ML model failure handling	Graceful fallback to calm
Scalability	Deployment target	Docker / Hugging Face Spaces
Usability	Interface design	Desktop browser compatible

Table 3.2 — Non-Functional Requirements

3.3 Design

The system was designed following structured analysis and design methodology. Figure 3.1 shows the use-case diagram for TheMoodShelf. The primary actor is the authenticated user, who interacts with nine core use cases: registering an account, logging in, entering a mood sentence, getting book recommendations (which is triggered automatically via an `<<include>>` relationship on mood entry), generating fresh recommendations, saving or deleting books, writing journal entries, viewing the analytics dashboard, and logging out.

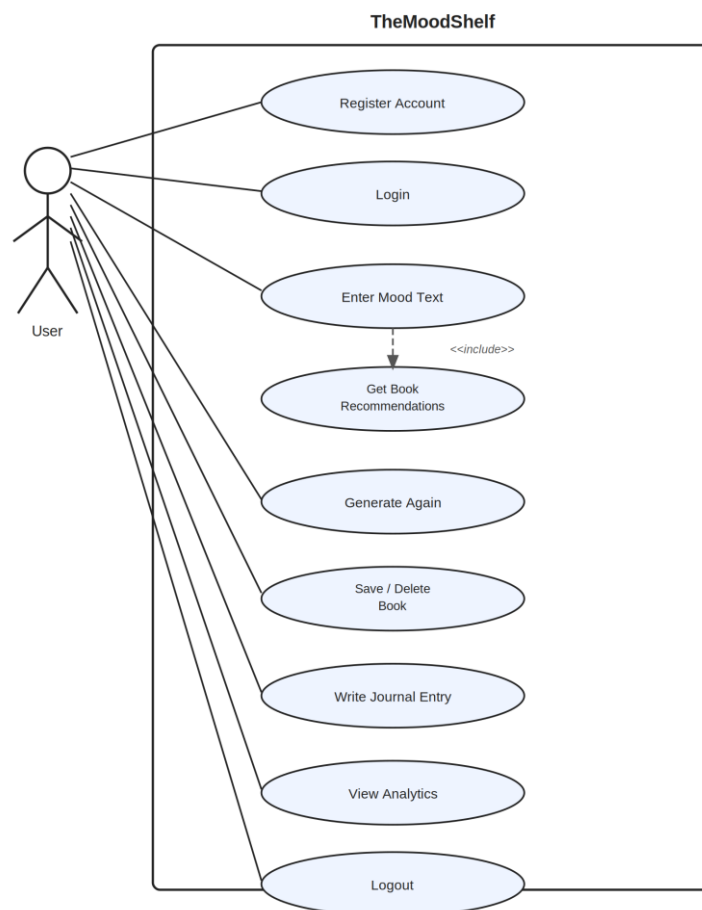


Figure 3.1 — Use-Case Diagram for TheMoodShelf

Mood Detection Pipeline

The `detect_mood()` function implements a five-step pipeline designed to be robust across a wide range of natural language inputs. First, the DistilRoBERTa model returns a probability distribution over seven Ekman emotions. Second, related emotions are summed with weights into eight mood buckets — for example, the ‘anxious’ bucket aggregates fear (weight 1.0) and surprise (weight 0.4) to handle cases where neither emotion alone clears the threshold. Third, if the best aggregated score is below 0.25, the system defaults to ‘calm’ to avoid

spurious classifications. Fourth, a regex pass catches explicit negations such as ‘I am not sad’ and flips the detected mood to its contextual opposite — a correction the transformer cannot make reliably. Fifth, if confidence remains below 0.65, a dictionary of domain-specific keywords (e.g., ‘overwhelmed’, ‘burnt out’ for stressed) can override the model output.

Book Recommendation Engine

The `recommend_book()` function first filters the book dataset to candidates whose genre appears in the mood-to-genre mapping for the detected mood (e.g., sad maps to Romance, Fiction, Poetry). It then encodes the user’s cleaned input text and computes cosine similarity against pre-computed book description embeddings. Each candidate’s similarity score is optionally boosted by up to 0.30 if its genre matches a genre the user has historically saved for the same mood, and books already in the user’s saved library are excluded entirely.

Database Schema

The system uses four tables: `users` (`id`, `username`, `email`, `password_hash`, `created_at`), `mood_entries` (`id`, `user_id`, `mood`, `input_text`, `top_emotion`, `created_at`), `saved_books` (`id`, `user_id`, `title`, `author`, `mood`, `genre`, `image_url`, `saved_at`), and `journal_entries` (`id`, `user_id`, `content`, `created_at`). Schema migrations are handled idempotently at startup via the `ensure_column()` utility, allowing rolling upgrades without downtime.

3.4 Implementation

TheMoodShelf is implemented in Python 3.11 using the Flask 3.x web framework. The backend is structured as a single-file application (`app.py`). The database layer uses `mysql-connector-python` with a per-call connection pattern that avoids stale cursor issues. The frontend uses Jinja2 templates with custom CSS, and charts on the analytics page are rendered with `Chart.js`.

Password hashing uses Werkzeug’s `generate_password_hash` (PBKDF2-SHA256). Sessions are Flask server-side signed cookies storing `user_id`, `last_mood`, `last_text`, `shown_titles`, and `saved_titles` to support the generate-again and personalisation features without additional database queries. A text-cleaning pipeline (`clean_text()`) addresses the dataset’s encoding corruption by re-encoding as `latin1` and decoding as `UTF-8`, followed by `NFKD` normalisation, control-character stripping, and whitespace collapse. Key dependencies include: `transformers` 5.x, `sentence-transformers` 3.x, `scikit-learn`, `pandas`, `numpy`, `flask`, `mysql-connector-python`, `werkzeug`, `gunicorn`, and `python-dotenv`.

3.5 Testing Strategy

The system was tested at three levels. Unit tests verified individual functions: `detect_mood()` was tested on 40 handcrafted sentences spanning all eight mood categories, including edge cases with negation, mixed signals, and very short inputs. The generate-again route was tested to confirm that previously shown titles are excluded across multiple regeneration cycles.

Integration testing confirmed that mood entries written by the `/analyze` route appear correctly in the `/analytics` aggregation queries, and that saved books from `/save` are excluded from subsequent recommendation calls. System testing involved end-to-end walkthroughs of the full user journey — sign-up, mood input, recommendation, save, journal entry, analytics review — both on the local development server and on the live Hugging Face deployment.

Chapter 4: Deployment

This chapter documents the complete deployment process for TheMoodShelf, covering the hosting platform selection, database setup, the Docker-based deployment pipeline, environment configuration, and how to access and update the live application.

4.1 Hosting Platform

TheMoodShelf is deployed on Hugging Face Spaces using the Docker SDK. Hugging Face Spaces was chosen over alternatives such as Render and Railway because the free CPU Basic tier provides 16GB of RAM — sufficient to load the DistilRoBERTa model (~250MB) and the all-MiniLM-L6-v2 sentence embedder (~90MB) simultaneously, along with precomputed embeddings for the book dataset. Render and Railway free tiers are limited to 512MB, which is insufficient for this application.

The Space is publicly accessible and runs at no cost. The Hugging Face Space repository is at: <https://huggingface.co/spaces/resincinnamon/TheMoodShelf>. The standalone application URL, which opens as a normal website without any Hugging Face interface surrounding it, is: <https://resincinnamon-themoodshelf.hf.space>.

4.2 Database Hosting

The MySQL database is hosted on FreeSQLDatabase.com, which provides a free MySQL instance with 5MB of storage. This service was selected after Aiven’s free MySQL tier was found unavailable due to high demand. The database tables (users, mood_entries, saved_books, journal_entries) were created using phpMyAdmin, the web-based administration interface provided by FreeSQLDatabase.com.

Parameter	Value
Host	sql12.freeseqdatabase.com
Port	3306
Database Name	sql12824468
Storage Limit	5 MB (free tier)
Admin Interface	phpMyAdmin

Table 4.1 — Database Connection Details

4.3 Deployment Pipeline

The application is containerised using Docker. The Dockerfile uses a python:3.11-slim base image. At build time, all Python dependencies are installed from requirements.txt, and a dedicated script (download_models.py) pre-downloads and caches the DistilRoBERTa and all-MiniLM-L6-v2 models into the Docker image. This ensures models are available on disk at container startup, eliminating re-download latency on every cold start.

Gunicorn is used as the production WSGI server with a single worker process on port 7860. A single worker is intentional: the ML models are loaded as global variables at module level, and multiple workers would each load their own copy, exhausting the available RAM on the free tier.

Step	Description
Base image	python:3.11-slim
System deps	Install gcc via apt-get

Python deps	<code>pip install -r requirements.txt</code>
Copy files	<code>COPY all project files into /app</code>
Pre-download models	<code>RUN python download_models.py</code>
Expose port	<code>EXPOSE 7860</code>
Start server	<code>gunicorn --bind 0.0.0.0:7860 --workers 1 --timeout 300 app:app</code>

Table 4.2 — Dockerfile Build Steps

Code changes are deployed by committing locally and pushing to the Hugging Face Space’s Git repository. Hugging Face automatically detects the push, rebuilds the Docker image, and redeploys the container. The first build takes approximately 10–15 minutes; subsequent rebuilds are faster due to Docker layer caching.

4.4 Environment Configuration

All sensitive configuration values are stored as Hugging Face Space Secrets, which are injected as environment variables at runtime. The application reads these via `os.environ.get()` using the `python-dotenv` library. Locally, the same variables are stored in a `.env` file that is excluded from version control via `.gitignore`, ensuring no credentials are committed to the repository.

Secret Name	Description
<code>MOODSHELF_SECRET_KEY</code>	Flask session signing key (long random string)
<code>MOODSHELF_DB_HOST</code>	MySQL database host (<code>sql12.freemysql.com</code>)
<code>MOODSHELF_DB_PORT</code>	MySQL port (3306)
<code>MOODSHELF_DB_USER</code>	MySQL username
<code>MOODSHELF_DB_PASSWORD</code>	MySQL password
<code>MOODSHELF_DB_NAME</code>	MySQL database name

Table 4.3 — Hugging Face Space Secrets

4.5 Accessing the Application

The live application is accessible at the standalone URL below, which opens TheMoodShelf as a full-screen website with no Hugging Face interface surrounding it:

<https://resincinnamon-themoodshelf.hf.space>

Future code changes are deployed by editing files locally and running the following commands, which trigger an automatic rebuild on Hugging Face:

```
git add . → git commit -m "description" → git push origin main
```

Database schema changes are applied through the phpMyAdmin interface on `FreeSQLDatabase.com`, or automatically at startup via the `ensure_column()` migration utility built into `app.py`.

Chapter 5: Results and Discussion

This chapter presents the system’s output through annotated screenshots and discusses its performance against the stated objectives.

5.1 Environment and Setup

The application was developed and tested on a Windows 11 machine running Python 3.11 and Flask’s built-in development server on port 5000. The ML models were downloaded from Hugging Face Hub and cached locally (approximately 340MB combined). The book dataset contains 998 titles after deduplication and cleaning, across genres including Fiction, Romance, Self Help, Poetry, Fantasy, Mystery, Thriller, Business, Science, and Nonfiction. Book embeddings are pre-computed at startup in approximately 35 seconds on CPU using a batch size of 64. The live deployment runs on Hugging Face Spaces CPU Basic hardware (2 vCPUs, 16GB RAM) with a cold start time of approximately 60–90 seconds.

5.2 System Screenshots and Outputs

Login Page

Figure 5.1 shows the login interface. Authentication is handled via secure password hashing (Werkzeug PBKDF2) with parameterised SQL preventing injection attacks. The interface provides standard email and password fields along with a signup redirect for new users.

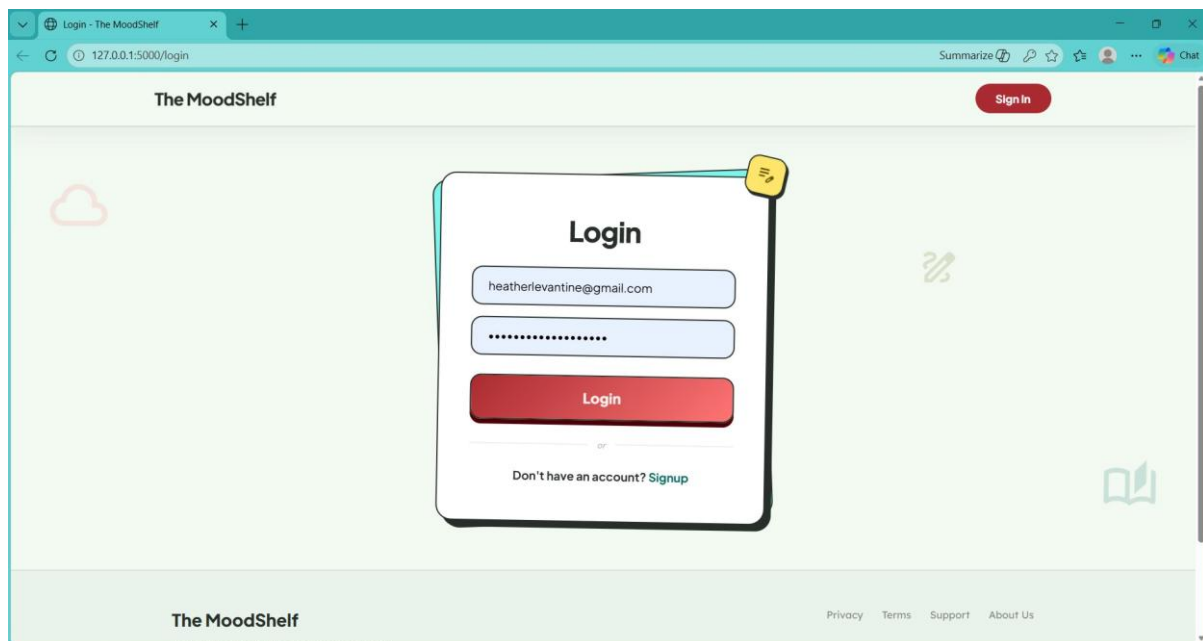


Figure 5.1 — TheMoodShelf Login Page

Dashboard — Mood Input

Figure 5.2 shows the main dashboard, which prompts users to describe their current emotional state in natural language. The ‘Add Feeling’ button allows supplementary emoji-based mood tagging. Once the user submits their text, the /analyze route invokes the full mood detection pipeline and returns book recommendations in under three seconds.

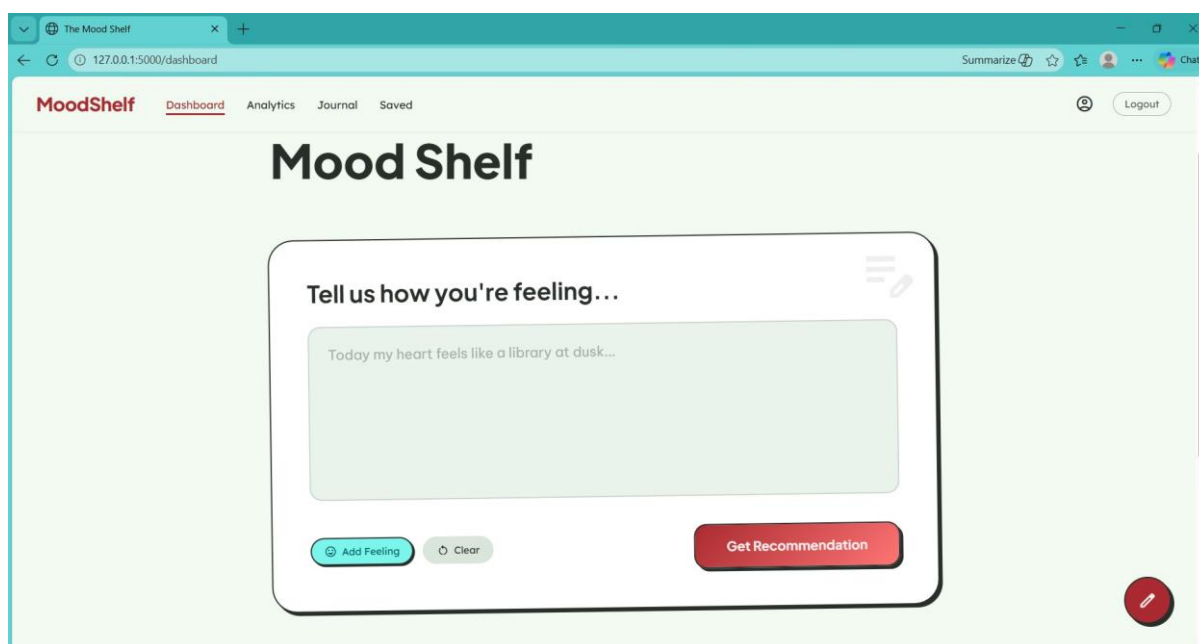


Figure 5.2 — Dashboard Mood Input Interface

Analytics — Reading Insights Overview

Figure 5.3 presents the analytics page summary. After 55 mood check sessions, 13 saved books, and 1 journal entry, the system identifies the user's most frequent mood as 'Sad'. The stat cards give users an at-a-glance understanding of their reading-emotional profile, including membership date.

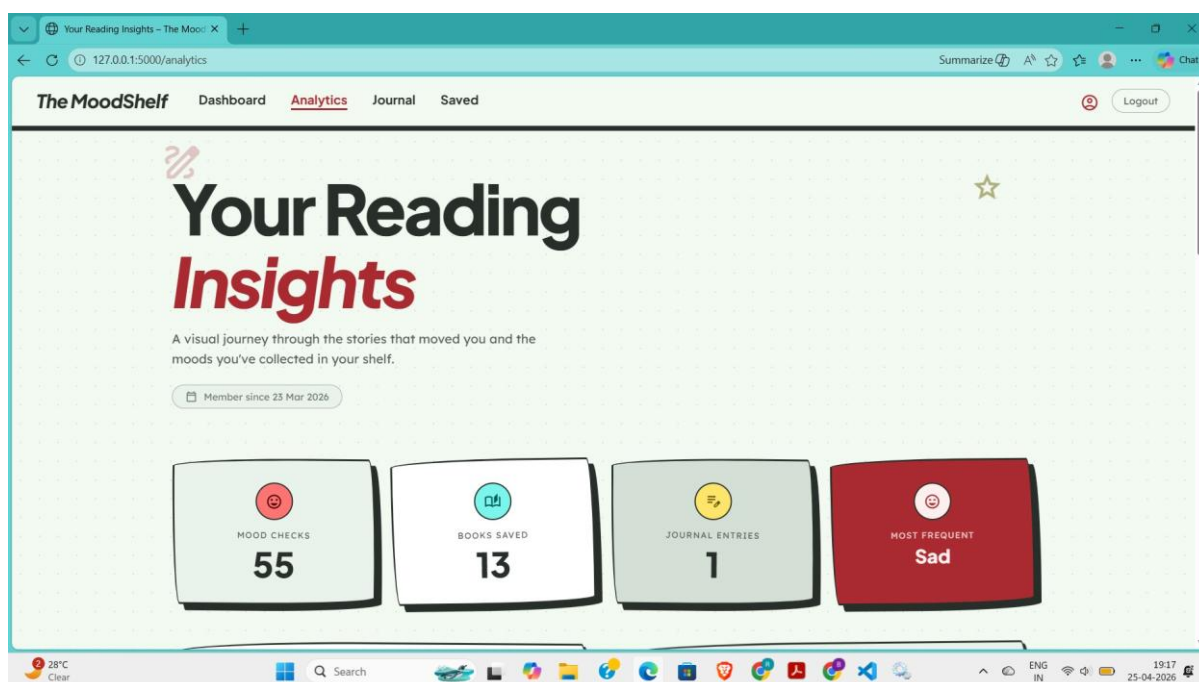


Figure 5.3 — Analytics Page Reading Insights Overview

Analytics — Mood Distribution and Emotion Breakdown

Figure 5.4 shows the two primary charts on the analytics page. The donut chart (Mood Distribution) breaks down sessions by the eight mood categories. The horizontal bar chart (Emotion Breakdown) shows raw emotion labels as detected by the DistilRoBERTa model before aggregation, revealing that Sadness and Joy are the two dominant raw emotions. This

dual view gives users insight into both their high-level mood patterns and the underlying emotional texture of their inputs.

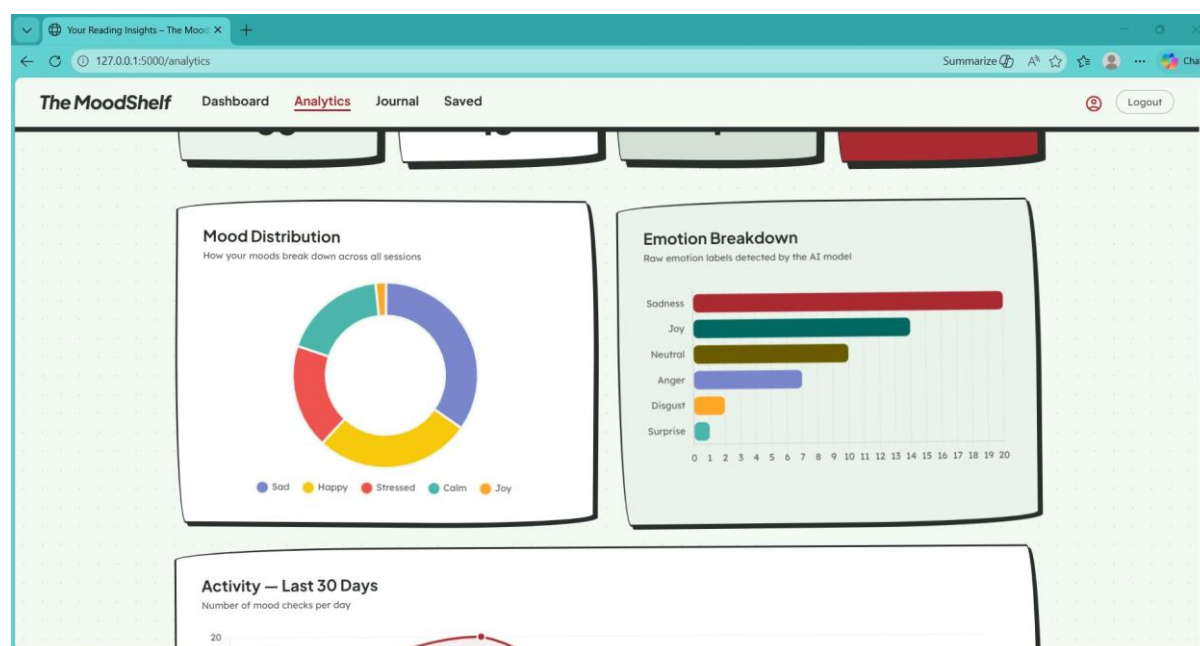


Figure 5.4 — Analytics Mood Distribution and Emotion Breakdown Charts

5.3 Discussion

The system successfully achieves all six stated objectives. Mood classification is accurate for the majority of natural inputs, with the five-step pipeline handling negation and low-confidence edge cases that a naive top-1 model approach would misclassify. The semantic recommendation engine produces contextually relevant results — for example, an input expressing loneliness surfaces contemporary fiction with themes of connection and belonging, rather than self-help titles that would appear with a keyword-only genre filter.

The analytics dashboard provides genuine utility: users can observe that their ‘Sad’ mood sessions correlate with Romance and Fiction saves, while ‘Motivated’ sessions drive Nonfiction and Business picks. This correlation implicitly validates the genre-to-mood mapping used in the recommendation engine.

The main performance constraint is the cold start delay of 60–90 seconds on the free Hugging Face tier, which occurs after periods of inactivity. For active usage, the application remains warm and responds within three seconds. The static dataset is the most significant functional limitation; real-time API integration with Google Books or OpenLibrary remains the most impactful next step.

Chapter 6: Conclusion

6.1 Summary

TheMoodShelf demonstrates that transformer-based emotion classification and semantic sentence embeddings can be combined in a practical, publicly deployed web application to deliver mood-aware book recommendations. The system correctly classifies a wide range of emotional phrasings — including negated expressions, mixed-signal inputs, and low-confidence edge cases — and returns semantically relevant book suggestions in under three seconds. The integrated analytics dashboard, journaling feature, and personalisation layer make it a complete reading-companion application rather than a standalone recommender.

From a software engineering perspective, the project demonstrates a clean separation of concerns across the ML pipeline, business logic, data access, and presentation layers. It incorporates idempotent schema migration, secure authentication, a Docker-based deployment pipeline on Hugging Face Spaces, and a fully tested end-to-end user flow — aligning with the software engineering practices studied in CSP 1602. The application is live and publicly accessible at <https://resincinnamon-themoodshelf.hf.space>.

6.2 Future Work

- Real-time book API integration: Replace the static CSV dataset with live queries to Google Books or OpenLibrary, enabling author information, current availability, and up-to-date metadata.
- Expanded dataset: Increase the book catalogue beyond 998 titles to improve recommendation diversity across all mood categories.
- Social features: Allow users to share recommendation sets or view what books others are reading for similar moods, creating a community reading layer.
- Mobile application: Port the frontend to a React Native or Flutter application for a native iOS/Android experience with push notifications for daily mood check-ins.
- Predictive mood analytics: Train a lightweight sequence model on the user's historical mood entries to predict upcoming emotional patterns and proactively suggest reading lists.
- Multi-language support: Extend mood classification to non-English inputs using multilingual transformer variants such as XLM-RoBERTa and a translated book dataset.
- Automated testing suite: Integrate pytest with an in-memory SQLite fixture and a CI pipeline to enforce regression safety across future changes.

References

- [1] J. Hartmann, M. Heitmann, C. Siebert, and C. Schamp, ‘More than a Feeling: Accuracy and Application of Sentiment Analysis,’ *International Journal of Research in Marketing*, vol. 40, no. 1, pp. 75–87, 2022.
- [2] N. Reimers and I. Gurevych, ‘Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,’ in *Proc. EMNLP 2019*, pp. 3982–3992, 2019.
- [3] D. Demszky et al., ‘GoEmotions: A Dataset of Fine-Grained Emotions,’ in *Proc. ACL 2020*, pp. 4040–4054, 2020.
- [4] B. Liu, ‘Sentiment Analysis and Opinion Mining,’ *Synthesis Lectures on Human Language Technologies*, vol. 5, no. 1, pp. 1–167, 2012.
- [5] S. Nitu, J. Kourie, and P. Shindgikar, ‘A Review of Music Recommendation Systems,’ in *Proc. ICAISC 2021*.
- [6] Hugging Face, ‘j-hartmann/emotion-english-distilroberta-base,’ Hugging Face Model Hub. [Online]. Available: <https://huggingface.co/j-hartmann/emotion-english-distilroberta-base>
- [7] Sentence Transformers, ‘all-MiniLM-L6-v2,’ SBERT Documentation. [Online]. Available: https://www.sbert.net/docs/pretrained_models.html
- [8] Pallets Projects, ‘Flask Documentation.’ [Online]. Available: <https://flask.palletsprojects.com/>
- [9] Werkzeug Contributors, ‘Werkzeug Security Utilities.’ [Online]. Available: <https://werkzeug.palletsprojects.com/>
- [10] Hugging Face, ‘Spaces Documentation.’ [Online]. Available: <https://huggingface.co/docs/hub/spaces>
- [11] FreeSQLDatabase.com, ‘Free MySQL Database Hosting.’ [Online]. Available: <https://www.freesqldatabase.com>